Number: P3309R0
Date: 2024-05-22
Audience: SG1 & LEWG
Reply-to: Hana Dusíková

Table of contents

constexpr atomic<T> and atomic_ref<T>

Introduction and motivation

This paper proposes marking most of `atomic<T>` methods and associated functions `constexpr` to allow usage of atomic code without changes in a constexpr and consteval code.

Proposed changes will allow implementing other types (`std::shared_ptr<T>`, persistent data structures with atomic pointers) and algorithms (thread safe data-processing, like scanning data with atomic counter) with just sprinkling `constexpr` to their specification.

Intention for wording changes

Mark all functions in [atomics] constexpr excluding all `volatile` overloads and all `wait` and `notify` functions. As all these can be implemented using `if consteval`:

```
template<class T>
constexpr T atomic_fetch_add(atomic<T
  if consteval {
    const auto previous = target->val
    target->value += diff;
    return previous;
  } else {
    __atomic_fetch_add(target->value,
  }
}
```

Synchronization functions and helpers